

## 02 — Writing Verbs

---

A verb is a Python file. When a player types a command that resolves to a verb, the engine builds a namespace, preprocesses the source, and executes it. This document is the reference for what you can rely on inside that file.

- [The shape of a verb](#)
- [The verb namespace](#)
- [The source preprocessor](#)
- [Matching objects](#)
- [Verb types: customizing the parser](#)
- [Messaging and emit substitution](#)
- [Calling other verbs](#)
- [Creating and changing objects](#)
- [Tasks: suspend and fork](#)
- [Interactive verbs with `yield`](#)
- [Permission-checked attribute access](#)
- [Conventions and idioms](#)

---

### The shape of a verb

---

A verb file is bare Python — no function definition, no imports, no boilerplate. It runs top to bottom and may `return` at any point (the engine wraps the body in a function so top-level `return` is legal). Standard library `import` works when you need it, but most verbs need nothing beyond the injected namespace.

```
"""
Optional docstring. The first lines often serve as the in-game help text.

Usage: <verb> <args>
"""

if not args:
    pobj.msg("Do what?")
    return

# ... logic ...
```

The leading triple-quoted docstring is conventional: it documents usage and is where in-game help comes from.

---

## The verb namespace

---

The namespace is assembled by `build_verb_namespace()` in `moo/verb_namespace.py`. Everything below is available as a bare name inside any verb.

### Core context

| Name                                    | What it is                                                                                                 |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>pobj</code> / <code>player</code> | The acting player object (the two names are the same object).                                              |
| <code>this</code>                       | The object the verb is <b>defined on</b> . Use it for the verb's own properties and helpers.               |
| <code>caller</code>                     | The object that invoked this verb via <code>call_verb</code> ( <code>None</code> for a top-level command). |
| <code>db</code>                         | The database singleton. <code>db.get_object(N)</code> fetches by number.                                   |
| <code>location</code>                   | The player's current location (or <code>None</code> ).                                                     |
| <code>verb</code>                       | The verb name as matched, e.g. <code>'look'</code> .                                                       |
| <code>args</code>                       | The argument string, stripped.                                                                             |
| <code>argstr</code>                     | The raw, unstripped argument string as typed.                                                              |

`pobj` vs `this` is the distinction that trips people up first: in `moo/verbs/17/jump.py`, `pobj` is the player jumping and `this` is the `ICRoom` the verb lives on. In a verb defined on a sword, `this` is the sword and `pobj` is whoever swung it.

## Parsed command parts

| Name                             | What it is                                                                 |
|----------------------------------|----------------------------------------------------------------------------|
| <code>dobj / dobjstr</code>      | Direct object: the matched object (or object number), and the raw text.    |
| <code>iobj / iobjstr</code>      | Indirect object: matched object and raw text.                              |
| <code>prep / preplist</code>     | The preposition ( 'in' , 'on' , 'from' , ...) and its tokens.              |
| <code>dobjlist / iobjlist</code> | The dobj/iobj strings split into tokens.                                   |
| <code>dobj2 / prep2</code>       | Secondary object/preposition for multi-preposition commands.               |
| <code>lhs / rhs</code>           | Left/right of the preposition, for assignment-style verbs ( @desc x = y ). |
| <code>arglist</code>             | <code>argstr</code> split on whitespace.                                   |
| <code>switches</code>            | Slash switches: look/brief → ['brief'] .                                   |

For `put gem in box`, you get `dobjstr='gem'`, `prep='in'`, `iobjstr='box'`, and — once you match them — `dobj / iobj` as objects.

## Safe Python builtins

A curated subset of Python builtins is injected (`moo/verb_namespace.py`): `len`, `str`, `int`, `float`, `bool`, `list`, `dict`, `set`, `tuple`, `type`, `range`, `enumerate`, `zip`, `map`, `filter`, `reversed`, `sorted`, `sum`, `min`, `max`, `abs`, `round`, `all`, `any`, `isinstance`, `print`. (`open`, `exec`, `eval`, `__import__`, etc. are deliberately not in the default set.)

## The MOO builtin library

Injected from `moo/builtins.py`. Grouped by purpose:

**Objects:** `create(parent, owner)`, `recycle(obj)`, `valid(obj)`, `get_object(objnum)`, `move(obj, dest)`, `chparent(obj, new_parent)`, `max_object()`.

**Properties:** `add_property(obj, name, value=None, perms='rc')`, `delete_property(obj, name)`, `properties(obj)`.

**Verbs:** `add_verb(obj, names, perms='rx', ...)`, `delete_verb(obj, name)`, `verbs(obj)`.

**Calling & scheduling:** `call_verb(obj, verb_name, ..., **kwargs)`, `suspend(seconds)`, `fork(...)`.

**Messaging:** `obj.msg(message, ...)` and `room.msg_room(message, ...)` are the verbs you call from verb code (defined on #1, so every object has them); `broadcast(message, ...)` reaches everyone connected. See [Messaging](#).

**Matching:** `pmatch`, `bmatch`, `match`, `omatch`, plus helpers `strip_articles`, `parse_ordinal`, `name_match`, `adj_match`.

**Search:** `search(query, ...)` / `find(query, ...)`.

**Admin / lifecycle:** `auth_level(obj)`, `sync_auth_flags(obj)`, `puppet(target)`, `force(player, command)`, `fire_hook(...)`, and the ticker functions `ticker_add(interval, verb_name, obj, idstring='')` / `ticker_remove(...)` / `ticker_remove_all(obj)` — hooks and the ticker are covered in Engine Systems.

**Helper modules:** `su` (string utilities — emit/pronoun substitution), `ou` (object utilities — `make_room`, `make_object`, ...), `eu` / `_effects` (the effects manager), and `globals` (the `moo.globals` module of shared constants).

---

## The source preprocessor

---

Before compilation, verb source is rewritten by `preprocess_verb_code()` (`moo/verbs.py`). Two transformations matter:

### Reference rewriting

A single master regex scans the source and rewrites object and system references **only in bare code** — strings and comments are matched first and left untouched.

| You write                         | Becomes                                | Notes                                                                                                                                                          |
|-----------------------------------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>#42</code>                  | <code>db.get_object(42)</code>         | An object literal.                                                                                                                                             |
| <code>\$wearable</code>           | <code>db.get_object(0).wearable</code> | A property on the system object ( <code>#0</code> ) — see <a href="#">named constants</a> below.                                                               |
| <code>\$su</code>                 | <code>su</code>                        | The one special case: <code>SU</code> is in <code>_PYTHON_CONSTANTS</code> , so it maps to the injected namespace name rather than a <code>#0</code> property. |
| <code>"text with #5"</code>       | unchanged                              | Inside a string.                                                                                                                                               |
| <code># a comment about #1</code> | unchanged                              | A real comment ( <code>#</code> not followed by a digit).                                                                                                      |

The rewrite also reaches **inside f-string expressions**, so `f"You see {#5.name}"` correctly becomes `f"You see {db.get_object(5).name}"`. The distinction between “comment” and “object ref” is precisely “`#` followed by a digit is an object; otherwise it’s a comment,” which is why `#42` and `#note` coexist safely.

## Named object constants

The `$` prefix gives well-known objects readable names. Every `$name` (except the special-cased `$su`) rewrites to `db.get_object(0).name` — a property read on the **system object #0**. Because those properties store object numbers, `$name` resolves to a well-known object without hardcoding its number:

```
# Instead of memorizing that the wearable base is #35:
glove = create(parent=$wearable)      # → create(parent=db.get_object(0).wearable)
if dobj.parent == $container:         # readable, refactor-proof
    ...
$eu.trigger(pobj, 'poison', 5, 3)     # $eu → db.get_object(0).eu (the effects object,
                                     #53)
```

This is the same indirection LambdaMOO's `$foo` corewords provide: a layer of named aliases over raw object numbers, so verb code reads in terms of *roles* (“the wearable prototype”, “the effects utility”) rather than magic numbers, and a world can renumber its prototypes by re-pointing `#0`'s properties.

**Adding one:** set a property on `#0` to the target object number — `@adprop #0.weapon = 200` (or from verb code, `db.get_object(0).weapon = 200`). After that, `$weapon` resolves to `#200` everywhere. The authoritative list of constants is simply whatever properties exist on `#0`; inspect it live with `+props #0`. Use short, lowercase names that name the role, and avoid Python keywords or builtin names.

Note: `eu` (the effects manager) and `su / ou` (string and object utilities) are *also* injected directly into the namespace, so you can write `eu.trigger(...)` without the `$`. The `$` form is the portable one for referencing *prototype objects* by role.

## Function wrapping

The body is wrapped so `return` works at top level:

```
# what you write
pobj.msg("Hello")
result = "done"

# what runs
def _verb_():
    pobj.msg("Hello")
    result = "done"
result = _verb_()
```

The wrapper's return value is captured as `namespace['result']` and becomes the value `call_verb` hands back to a caller. Compiled code is cached, so the preprocessing cost is paid once per verb version, not per invocation.

---

## Matching objects

---

Turning "2nd blue sword" into the object the player means is the job of the matchers in `moo/match_utils.py`. Both major matchers resolve in the same order — possessives (`my sword` restricts to inventory), keywords (`me`, `here`), dbrefs (`#3`, `$utils`), then name matching against a candidate list.

| Function                                   | Returns                                                                                           | Use when                                              |
|--------------------------------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| <code>pmatch(inp, pobj, candidates)</code> | The single best match, or <code>None</code> .                                                     | <b>Player verbs.</b> This is the default.             |
| <code>bmatch(inp, pobj, candidates)</code> | A list of all matches.                                                                            | <b>Staff verbs</b> and anywhere you want every match. |
| <code>match(inp, candidates)</code>        | Name-only match (no keywords/refs).                                                               | Low-level matching against an explicit list.          |
| <code>omatch(inp, pobj)</code>             | Keyword/ref only ( <code>me</code> , <code>here</code> , <code>#N</code> , <code>\$name</code> ). | When you only want references, not names.             |

Name matching understands how players actually talk:

- **Articles** are stripped (`the`, `a`, `an`, `some`).
- **Ordinals** select among duplicates: `2nd sword`, `third rope`, or a bare `3`.
- **Adjectives** filter in order: in `blue sword`, `sword` is the noun and `blue` must appear (as an in-order substring of the name, or in the object's `adjectives` list).
- **Aliases** and the atomic `noun` property are both checked, with multi-character tokens matched by prefix.

A standard player-verb matching block:

```
target = pmatch(dobj, pobj, list(pobj.location.contents))
if not target or not getattr(target, 'existent', False):
    pobj.msg("You don't see that here.")
    return
```

The `existent` check after matching is a project convention for **unhidden player verbs**: confirm the matched object still exists and is real before acting on it. Staff verbs on #3 use `bmatch` and generally skip that check.

## Verb types: customizing the parser

The parsed command parts your verb reads ( `dobj` , `prep` , `iobj` , `switches` , `lhs / rhs` , ...) don't appear by magic — they're produced by the verb's **verb type**, a small, swappable class that owns the *parse* step. Every verb has one, named by its `parent_type` (default `moo.verb_types.MasterVerb`). Coders can replace it, so a verb can parse its argument string however it likes. The classes live in `moo/verb_types.py`.

### How a verb runs

Before your verb's code executes, the engine ( `moo/verb_namespace.py` ) resolves the verb's `parent_type` to a class, instantiates it, sets the runtime context ( `pobj` , `args` , `cmdstring` , `switches`...), and calls these lifecycle hooks in order:

1. `at_pre_cmd()` — pre-parse setup (e.g. an early permission check).
2. `parse()` — turn `self.args` into structured attributes.
3. **your verb body** — runs with those attributes available in its namespace.
4. `at_post_cmd()` — cleanup / logging / side effects.

The attributes `parse()` populates on the instance are exactly what [the namespace](#) exposes to your code. If the verb type can't be instantiated or `parse()` raises, the engine falls back to a default parse so the verb still gets sensible `dobj` / `prep` / `iobj` values.

### The two built-in types

`BaseVerb`          Minimal — `parse()` is a no-op; `self.args` is the raw string.  
└─ `MasterVerb` Standard `dobj` / `prep` / `iobj` / `switches` parsing (the default).

- **MasterVerb** does classic LambdaMOO-style parsing: it pulls MUSH switches off the verb name, finds the first preposition, and splits the rest into `dobj` / `prep` / `iobj` (plus `lhs / rhs` , a second preposition, and the list forms). This is what almost every verb wants.
- **BaseVerb** is “bring your own parser”: its `parse()` does nothing, so `self.args` is just the raw argument string for you to interpret. Use it for verbs whose syntax doesn't fit the `dobj/prep/iobj` mould.

### Three levels of customization

**1. Swap the preposition regex ( `rexp` ).** `MasterVerb.parse()` uses the verb's `rexp` instead of the global `PREP_REGEX` when one is set — the lightest way to change which words/separators split `dobj` from `iobj` .

**2. Override `parse()` .** Take full control of how `self.args` becomes structured attributes. Set whatever your verb body will read:

```

from moo.verb_types import MasterVerb, register_verb_type

@register_verb_type
class KeyValueVerb(MasterVerb):
    """Parse 'name: value' pairs instead of dobj/prep/iobj."""
    def parse(self):
        key, _, val = self.args.partition(':')
        self.field = key.strip()
        self.value = val.strip()

```

**3. Start from `BaseVerb`** . When there's no standard structure at all, inherit `BaseVerb` and parse `self.args` yourself.

## Registering and attaching a custom type

Verb types live in a registry keyed by dotted path. Register a class with the `@register_verb_type` decorator (shown above); the engine resolves it later via `resolve_verb_type(path)` . Attach it to a verb by setting its `parent_type` :

```

# From verb code, attach a registered type to a new verb:
add_verb(this, ['setfield'], parent_type='yourmodule.KeyValueVerb')

```

`define_verb(key, parent='...', rexp=..., parse=..., func=...)` is the factory the engine uses to build verb classes at runtime (hot-reload, in-game creation); it takes the same `parse` / `rexp` / `parent` knobs directly.

In-game, `@adverb ... base` selects `BaseVerb` instead of the default `MasterVerb` (see Building Worlds); attaching an arbitrary *custom* registered type is done from verb code with `add_verb(..., parent_type=...)` or `define_verb(...)` .

## Messaging and emit substitution

There are three ways to put text in front of players, all of which run the text through the substitution engine (`moo/string_utils.py`) and then the recipient's color/wrap/screenreader pipeline:

- `pobj.msg("...")` — to one player.
- `room.msg_room("...", exclude=[pobj], sub=pobj, dob=target)` — to everyone in a room, optionally excluding actors.
- `broadcast("...")` — to everyone connected.

`msg` and `msg_room` are themselves verbs defined on #1 (`Root_Object`), so every object inherits them and verb code always reaches for `obj.msg(...)` rather than a free function. (Under the

hood `msg` wraps a lower-level `notify()` primitive, but you write `obj.msg(...)` ; because it's a verb, an object can even override it to filter or redirect its own messages — e.g. a deafened character.)

## Substitution tokens

Pass the actors as `sub` (subject), `dob` (direct object), `iob` (indirect object), `uob` (noun source); the tokens then render per recipient:

| Token                            | Renders                                                                                                |
|----------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>%s / %S</code>             | subject name / capitalized name                                                                        |
| <code>%d / %D</code>             | direct-object name / capitalized                                                                       |
| <code>%i / %I</code>             | indirect-object name / capitalized                                                                     |
| <code>%u / %U</code>             | noun (from <code>uob</code> ) / capitalized                                                            |
| <code>%ps %po %pp %pa %pr</code> | gender pronouns of the subject: subjective / objective / possessive / possessive-adjective / reflexive |

Both `%` and `$` prefixes are accepted for these tokens. Capitalize the token to capitalize the output (`%Ps` → “He”). Gender comes from the actor’s `gender` property; the four supported genders (`male`, `female`, `neutral`, `ambiguous`) map to pronoun sets in `moo/globals.py`, with `ambiguous` using singular *they*.

A canonical three-audience emit, from `moo verbs/17/tap.py` :

```
pobj.msg("You tap %d on the shoulder.", dob=tobj)
tobj.msg("%S taps you on the shoulder.", sub=pobj)
pobj.location.msg_room("%S taps %d on the shoulder.",
                       exclude=[pobj, tobj], sub=pobj, dob=tobj)
```

The actor sees “You tap Bob...”, Bob sees “Alice taps you...”, and the room sees “Alice taps Bob...” — one logical event, correctly conjugated for each viewer.

## Pronoun helpers

For longer narrative strings, `su.psub1(text, eobj=...)` substitutes a single actor’s pronouns (`%N / %CN` name, `%EPS / %EP0 / %EPP / %EPR` and capitalized variants), and `su.psub2(text, eobj=..., tobj=...)` adds a target’s pronouns (`%T / %CT`, `%OPS / %OP0 / ...`). Reach for these whenever a verb narrates an interaction between two characters.

## Calling other verbs

---

`call_verb(obj, verb_name, **kwargs)` runs another verb and returns its result. Extra keyword arguments are injected into the called verb's namespace, which is the standard way to pass structured data between verbs:

```
# A player verb on the room dispatches to the object's implementation:
call_verb(exit, 'invoke')

# Pass structured data into the called verb's namespace:
call_verb(this, 'on_use', actor=pobj, target=dobj)
```

Switch syntax works here too: `call_verb(obj, 'look/brief')`. Nested calls are bounded by the maximum stack depth (50) to prevent runaway recursion.

This dispatch pattern is pervasive: room-level verbs are thin and call object-specific `_`-suffixed implementations. The room's `sit` verb matches the furniture and calls `sit_` on it; `get` / `put` / `look` call the `in_` / `on_` / `under_` / `behind_` family on the target object. Keeping the player-facing verb a dispatcher lets each object type customize behavior without re-implementing parsing.

---

## Creating and changing objects

---

Verbs can build the world at runtime:

```
sword = create(parent=db.get_object(35))    # child of #35 (BaseWearable)
sword.noun = "sword"
sword.description = "A plain iron sword."
add_property(sword, 'damage', 12)
move(sword, pObj)                          # into the player's inventory
```

- `create(parent, owner)` returns a new `M00Object`.
- Property writes are just attribute assignment ( `sword.damage = 12` ), or `add_property` when you want explicit perms.
- `move(obj, dest)` relocates an object and fires the before/after move hooks.
- `recycle(obj)` deletes it (sending it to the trash).
- `chparent(obj, new_parent)` reparents.

The higher-level `ou` module ( `ou.make_room` , `ou.make_object` , ...) wraps these for common world-building shapes, and the `@`-command toolkit in Building Worlds is built on exactly these primitives.

---

## Tasks: suspend and fork

---

Because verbs are real code running in a shared world, every verb runs inside a task with limits ( `moo/tasks.py` ): a tick budget, a wall-clock budget, a verb stack depth of 50, and a fork depth of 10.

- **`suspend(seconds)`** parks the verb and resumes it automatically later. Use it for delays and pacing without blocking other players:

```
pobj.msg("You begin picking the lock...")
suspend(3)
pobj.msg("...click. The lock opens.")
```

- **`fork(...)`** schedules independent follow-up work that runs as its own task.
- **Round time** (the `rt` property) is the gameplay-level cooldown layered on top of this: most action verbs early-return if `pobj.rt > 0` and call a helper to set round time after acting. That is a game convention, distinct from the task scheduler's hard limits.

---

## Interactive verbs with `yield`

---

A verb that needs to ask the player something mid-execution becomes a generator by using `yield`. Each `yield "prompt"` sends the prompt and resumes the verb with the player's next line:

```
ans = yield "This will overwrite the existing inscription. Proceed? (y/n) "
ans = (ans or '').strip().lower()
if ans[:1] == 'y':
    this.inscription = new_text
    pobj.msg("You carve the new inscription.")
else:
    pobj.msg("You leave it as it was.")
```

The engine detects the generator return value and drives an interactive session automatically. Multi-step new-character setup and builder confirmations both use this. No callback plumbing required — the verb reads like a linear conversation.

---

## Permission-checked attribute access

---

The `getattr` / `setattr` injected into the namespace are **not** Python's builtins — they are wrappers (`moo/verb_namespace.py`) that enforce MOO property permissions when the target is a `M000bject`:

- **Read** is allowed if the property is readable (`r`), or the player owns it, or the player is a wizard.
- **Write** is allowed if the player owns the property and it is writable (`w`), or the player is a wizard.

Non-`M000bject` targets pass straight through. `hasattr` is the standard builtin and does *not* enforce permissions, which is why the idiom for “read a property that may be unset” is `getattr(obj, 'name', default)` rather than a `hasattr` guard:

```
rt = getattr(pobj, 'rt', None) or 0          # unset → 0
if getattr(exit, 'jumpable', False):        # unset → False
    ...
```

## Conventions and idioms

---

These are project conventions (see also the codebase memory and existing verbs), not engine requirements, but following them keeps new content consistent:

- **Define in-game commands on the room parents — #16 (OOC) and #17 (IC) — not on player/character objects.** Characters carry only staff verbs (from #3). To give an individual object custom behavior, have the room verb call a `<verb>_` hook on the matched object rather than adding a player verb to the object.
- **Player verbs use `pmatch` ; staff verbs (on #3) use `bmatch` .**
- **Send player output with `obj.msg(...)` / `obj.msg_room(...)` , not a raw `notify()` — `msg` is the inherited verb (on #1) every object exposes.**
- **Unhidden player verbs check `obj.existent` after matching** and treat a non-existent object as no match.
- **Pass actors explicitly to messaging:** `sub=pobj, dob=dobj, iob=iobj` so `%S / %d / %i` resolve. For `gmove` on exits, pass `sub=player, dob=this` (the exit) so `$d` substitutes to the exit.
- **Round-time gate first.** Action verbs check `pobj.rt / position / status` before doing work and set round time after.
- **Dispatch to `_`-suffixed implementations.** Keep the typed verb a thin matcher/dispatcher; put behavior on the target object's `verb_` method.
- **Hidden / internal verbs** are named with a trailing `_` (called programmatically) or a leading `_` (ticker callbacks, lifecycle hooks) and are marked hidden so players can't invoke them.

- **Colors:** prefer `%<245>` for dim UI chrome; avoid `%<240>` (too dark); `%n` resets. All of it disappears under screenreader mode, so never encode meaning in color alone.
- 

Next: Building Worlds — the in-game `@`-command toolkit that drives all of the above.